LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL 4

Singly Linked List



Disusun oleh:

Rafi Imam Nasrullah

109082530010

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

LATIHAN SOAL MODUL

Link Repo Github Modul X : LAPRAK_RAFI_STRUKTUR-DATA/MODUL 4 at main · RafiImamNasrullah/LAPRAK_RAFI_STRUKTUR-DATA

1. **Guide 1**

   Source Code

```
#include <iostream>

using namespace std;

typedef int infotype;
typedef struct elmlist *address;

struct elmlist {
   infotype info;
   address next;
};

struct List {
   address first;
};

void createList(List &L) {
   L.first = NULL;
}

address alokasi(infotype x) {
   address P = new elmlist;
   P->info = x;
   P->next = NULL;
   return P;
}

void dealokasi(address &P) {
   delete P;
   P = NULL;
}

void insertFirst(List &L, address P) {
   P->next = L.first;
   L.first = P;
}

void insertLast(List &L, address P) {
   if (L.first == NULL) {
      insertFirst(L, P);
   } else {
      address Q = L.first;
      while (Q->next != NULL) {
         Q = Q->next;
      }
```

```
      Q->next = P;
   }
}

void insertAfter(List &L, address P, address Prec) {
   if (Prec != NULL) {
      P->next = Prec->next;
      Prec->next = P;
   }
}

void deleteFirst(List &L, address &P) {
   if (L.first != NULL) {
      P = L.first;
      L.first = P->next;
      P->next = NULL;
   }
}

void deleteLast(List &L, address &P) {
   if (L.first != NULL) {
      if (L.first->next == NULL) {
         deleteFirst(L, P);
      } else {
         address Q = L.first;
         while (Q->next->next != NULL) {
            Q = Q->next;
         }
         P = Q->next;
         Q->next = NULL;
      }
   }
}

void deleteAfter(List &L, address &P, address Prec) {
   if (Prec != NULL && Prec->next != NULL) {
      P = Prec->next;
      Prec->next = P->next;
      P->next = NULL;
   }
}

void printInfo(List L) {
   address P = L.first;
   while (P != NULL) {
      cout << P->info << ' ';
      P = P->next;
   }
   cout << '\n';
}
```

```
int main() {
    List L;
    address P1, P2, P3, P4, P5;

    createList(L);

    P1 = alokasi(2);
    insertFirst(L, P1);

    P2 = alokasi(0);
    insertFirst(L, P2);

    P3 = alokasi(8);
    insertFirst(L, P3);

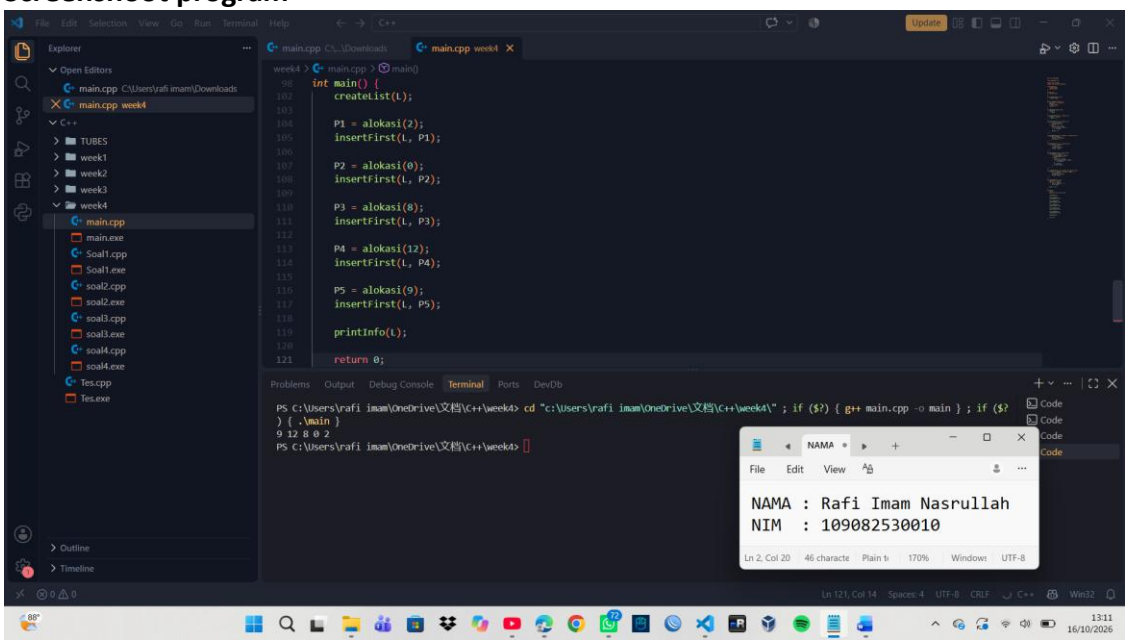
    P4 = alokasi(12);
    insertFirst(L, P4);

    P5 = alokasi(9);
    insertFirst(L, P5);

    printInfo(L);

    return 0;
}
```

Screenshoot program



Deskripsi program

Program ini merupakan program untuk membuat dan mengelola Singly Linked List menggunakan konsep ADT (Abstract Data Type). Pertama, fungsi createList() digunakan untuk menginisialisasi list dalam keadaan kosong. Selanjutnya, program membuat lima node dengan nilai 2, 0, 8, 12, dan 9 menggunakan fungsi alokasi(),

kemudian memasukkannya ke bagian depan list menggunakan fungsi insertFirst(). Karena setiap data dimasukkan di bagian depan, urutan data dalam list menjadi terbalik dari urutan pemasukan, yaitu 9 -> 12 -> 8 -> 0 -> 2. Terakhir, fungsi printInfo() menampilkan seluruh data dalam linked list sehingga output yang dihasilkan adalah 9 12 8 0 2.

2. **Soal 1**

Source Code

```
#include <iostream>
#include <string>
using namespace std;

struct Node {
   string nama;
   Node* next;
};

Node* head = nullptr;
Node* tail = nullptr;

void insertLast(const string& nama) {
   Node* baru = new Node{nama, nullptr};
   if (head == nullptr) {
      head = tail = baru;
   } else {
      tail->next = baru;
      tail = baru;
   }
}

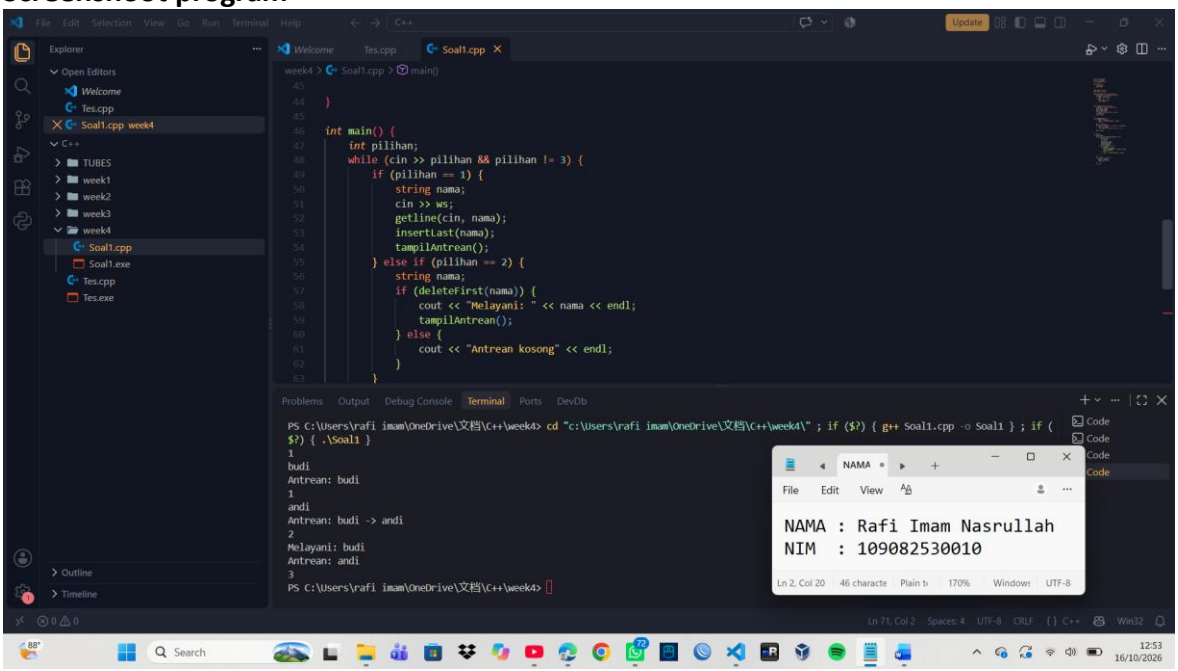
bool deleteFirst(string& nama) {
   if (head == nullptr) return false;
   Node* hapus = head;
   nama = hapus->nama;
   head = head->next;
   if (head == nullptr) tail = nullptr;
   delete hapus;
   return true;
}

void tampilAntrean() {
   if (head == nullptr) {
      cout << "Antrean: kosong" << endl;
```

```
        return;
    }
    cout << "Antrean: ";
    for (Node* p = head; p != nullptr; p = p->next) {
        cout << p->nama;
        if (p->next != nullptr) cout << " -> ";
    }
    cout << endl;
}

int main() {
    int pilihan;
    while (cin >> pilihan && pilihan != 3) {
        if (pilihan == 1) {
            string nama;
            cin >> ws;
            getline(cin, nama);
            insertLast(nama);
            tampilAntrean();
        } else if (pilihan == 2) {
            string nama;
            if (deleteFirst(nama)) {
                cout << "Melayani: " << nama << endl;
                tampilAntrean();
            } else {
                cout << "Antrean kosong" << endl;
            }
        }
    }
    while (head != nullptr) {
        Node* t = head;
        head = head->next;
        delete t;
    }
    return 0;
}
```

Screenshoot program



Deskripsi program

Program ini merupakan program antrean sederhana menggunakan struktur data Singly Linked List dengan prinsip FIFO (First In, First Out), yaitu orang yang pertama masuk akan dilayani terlebih dahulu. Ketika pengguna memasukkan angka 1 dan nama budi, program menambahkan Budi ke dalam antrean dan menampilkan Antrean: budi. Selanjutnya, pengguna memasukkan angka 1 dan nama andi, sehingga Andi ditambahkan di belakang Budi dan antrean menjadi Antrean: budi -> andi. Ketika pengguna memasukkan angka 2, program melayani Budi sebagai orang pertama dalam antrean, kemudian menghapusnya dan menampilkan Melayani: budi serta Antrean: andi. Terakhir, ketika pengguna memasukkan angka 3, program berhenti dan menghapus sisa data antrean yang masih tersimpan di memori.

3. Soal 2

Source Code

```
#include <iostream>
using namespace std;

struct Node {
   int data;
   Node* next;
};

Node* head = nullptr;
Node* tail = nullptr;
```

```
void insertLast(int x) {
   Node* baru = new Node{x, nullptr};
   if (head == nullptr) head = tail = baru;
   else { tail->next = baru; tail = baru; }
}

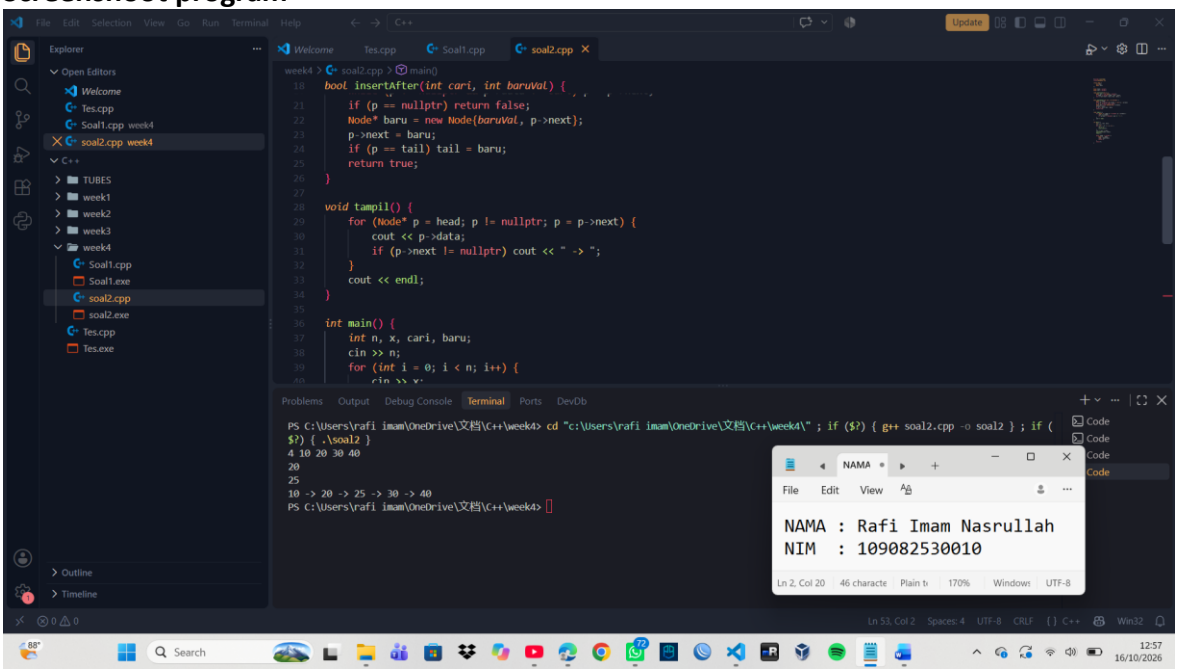
bool insertAfter(int cari, int baruVal) {
   Node* p = head;
   while (p != nullptr && p->data != cari) p = p->next;
   if (p == nullptr) return false;
   Node* baru = new Node{baruVal, p->next};
   p->next = baru;
   if (p == tail) tail = baru;
   return true;
}

void tampil() {
   for (Node* p = head; p != nullptr; p = p->next) {
      cout << p->data;
      if (p->next != nullptr) cout << " -> ";
   }
   cout << endl;
}

int main() {
   int n, x, cari, baru;
   cin >> n;
   for (int i = 0; i < n; i++) {
      cin >> x;
      insertLast(x);
   }
   cin >> cari >> baru;
   insertAfter(cari, baru);
   tampil();

   while (head != nullptr) {
      Node* t = head;
      head = head->next;
      delete t;
   }
   return 0;
}
```

Screenshoot program



Deskripsi program

Program ini merupakan program untuk menambahkan data setelah nilai tertentu menggunakan struktur data Singly Linked List. Pertama, pengguna memasukkan jumlah data sebanyak 4, kemudian memasukkan nilai 10, 20, 30, dan 40 sehingga terbentuk linked list 10 -> 20 -> 30 -> 40. Selanjutnya, pengguna memasukkan nilai 20 sebagai data yang dicari dan nilai 25 sebagai data baru yang akan ditambahkan setelahnya. Program mencari nilai 20, kemudian menyisipkan nilai 25 tepat setelah nilai tersebut sehingga linked list berubah menjadi 10 -> 20 -> 25 -> 30 -> 40. Setelah itu, program menampilkan seluruh data yang telah diperbarui dan menghapus semua node yang tersimpan di memori sebelum program berakhir.

4. Soal 3

Source Code

```
#include <iostream>
using namespace std;

struct Node {
   int data;
   Node* next;
};

Node* head = nullptr;
Node* tail = nullptr;

void insertLast(int x) {
```

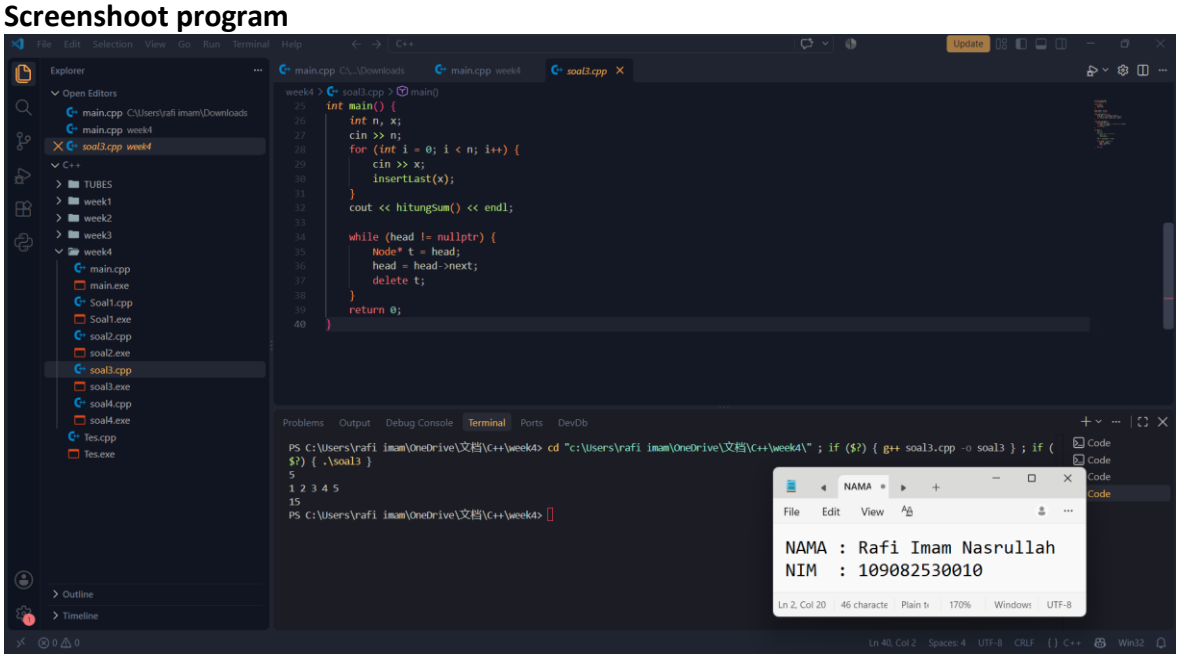
```
    Node* baru = new Node{x, nullptr};
    if (head == nullptr) head = tail = baru;
    else { tail->next = baru; tail = baru; }
}

long long hitungSum() {
    long long total = 0;
    for (Node* p = head; p != nullptr; p = p->next)
        total += p->data;
    return total;
}

int main() {
    int n, x;
    cin >> n;
    for (int i = 0; i < n; i++) {
        cin >> x;
        insertLast(x);
    }
    cout << hitungSum() << endl;

    while (head != nullptr) {
        Node* t = head;
        head = head->next;
        delete t;
    }
    return 0;
}
```

Screenshoot program



Deskripsi program

Program ini merupakan program untuk menghitung jumlah seluruh data dalam struktur Singly Linked List. Pertama, pengguna memasukkan jumlah data sebanyak 5, kemudian memasukkan nilai 1, 2, 3, 4, dan 5 yang disimpan ke dalam linked list secara berurutan. Setelah semua data dimasukkan, program memanggil fungsi hitungSum() untuk menjumlahkan seluruh nilai dalam linked list, yaitu 1 + 2 + 3 + 4 + 5, sehingga menghasilkan output 15. Setelah hasil penjumlahan ditampilkan, program menghapus seluruh node yang tersimpan di memori sebelum program berakhir.

5. **Soal 4**

Source Code

```
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

struct Buku {
   string judul;
   string tahun;
   Buku* next;
};

Buku* head = nullptr;
Buku* tail = nullptr;

void insertLast(const string& judul, const string& tahun) {
   Buku* baru = new Buku{judul, tahun, nullptr};
   if (head == nullptr) head = tail = baru;
   else { tail->next = baru; tail = baru; }
}

void deleteLast() {
   if (head == nullptr) return;
   if (head->next == nullptr) {
      delete head;
      head = tail = nullptr;
      return;
   }
```

```
    Buku* p = head;
    while (p->next != tail) p = p->next;
    delete tail;
    tail = p;
    tail->next = nullptr;
}

void tampil() {
    cout << "Daftar Buku:" << endl;
    for (Buku* p = head; p != nullptr; p = p->next)
        cout << p->judul << " (" << p->tahun << ")" << endl;
}

bool semuaAngka(const string& s) {
    if (s.empty()) return false;
    for (char c : s) if (!isdigit((unsigned char)c)) return false;
    return true;
}

int main() {
    string baris;
    int batas = -1;
    int terbaca = 0;
    bool pertama = true;

    while (getline(cin, baris)) {
        if (!baris.empty() && baris.back() == '\r') baris.pop_back();
        if (baris.empty()) break;

        if (pertama) {
            pertama = false;
            if (semuaAngka(baris)) {
                batas = stoi(baris);
                continue;
            }
        }

        size_t pos = baris.find_last_of(' ');
        if (pos != string::npos) {
```

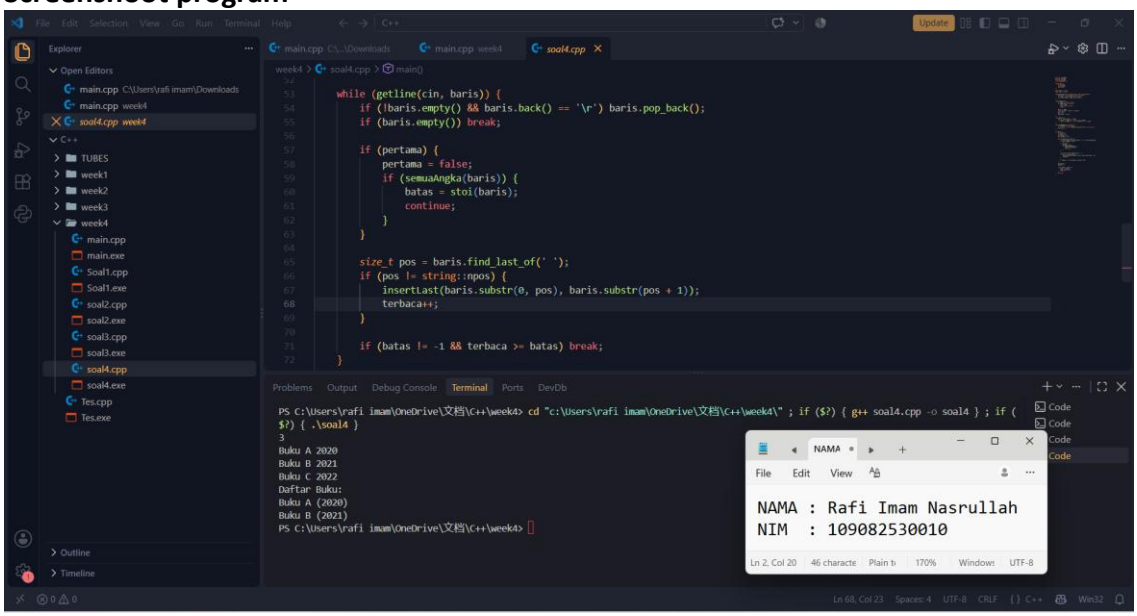
```
            insertLast(baris.substr(0, pos), baris.substr(pos + 1));
            terbaca++;
        }

        if (batas != -1 && terbaca >= batas) break;
    }

    deleteLast();
    tampil();

    while (head != nullptr) {
        Buku* t = head;
        head = head->next;
        delete t;
    }
    return 0;
}
```

Screenshoot program



Deskripsi program

Program ini merupakan program untuk mengelola daftar buku menggunakan struktur data Singly Linked List. Pengguna memasukkan angka 3 sebagai jumlah buku, kemudian memasukkan data Buku A tahun 2020, Buku B tahun 2021, dan Buku C tahun 2022. Setiap buku disimpan ke dalam linked list secara berurutan. Setelah semua data dimasukkan, fungsi deleteLast() menghapus buku terakhir, yaitu Buku C tahun 2022. Selanjutnya, fungsi tampil() menampilkan daftar buku yang tersisa, yaitu Buku A (2020) dan Buku B (2021). Setelah itu, program menghapus seluruh node yang tersisa dari memori sebelum program berakhir.